

Feature Engineering

⇒ What is Feature engineering? It's the process of preparing and transforming raw data into informative features that improves ML model performance.

⇒ It includes tasks such as:

- cleaning data
- Encoding categorical variables
- Scaling numerical values
- creating new features
- selecting most useful features

Handling Missing Values:

⇒ What problem it solves? Missing values (NaN, NULL, None) can:

- Reduce model accuracy
- Cause many ML algo. to fail
- Produce biased predictions

⇒ Why do missing values occur?

- Human error during data entry
- Users skip optional fields
- Sensor or device failure

⇒ When to handle missing values? we shall handle it before:

- Feature scaling
- Encoding
- Feature selection
- Model training

⇒ When not to use a particular method?

- Mean → skewed data or many outliers
- Mode → Numerical data (usually)
- Dropping rows or columns leads to missing data of large amount.

⇒ Types of Missing values:

- MCAR → Completely Random

→ drop rows (if any)
→ Mean/Median/Mode imputation

• MAR → depends on another feature

→ KNN
→ MICE
→ Related feature imputation

• MNAR → depends on missing value itself

→ domain knowledge
→ missing indicator

⇒ Methods:

Method	use when
drop rows	< 5% missing data
drop col ^m	> 60-70% missing and unimportant
mean	Numerical + Normal dist ⁿ
Median	Numerical + skewed/outliers
Mode	categorical
Constant ("unknown", 0)	Missing has meaning
Forward Fill	Time series
Backward Fill	Time series
KNN Imputer	correlated Features
Iterative (MICE)	Multiple correlated Features
Missing Indicator	Missingness itself is useful

⇒ Decision Flow:
Missing values?

→ < 5% → drop rows
→ > 70% col^m missing
 ↳ drop col^m
→ Numerical?
 ↳ Normal → Mean
 ↳ Skewed → Median
 ↳ Correlated → KNN/MICE
→ categorical?
 ↳ Few missing → Mode
 ↳ Unknown is meaningful → constant
 ↳ Missing is informative → Indicator

⇒ Algorithm Compatibility:

Handles missing values	Not handling missing values directly
XGBoost LightGBM CatBoost	LR LogisticR, SVM, KNN, Neural Networks, decision tree, Random forest

⇒ Real world Examples:

- Healthcare: Missing lab test
- Banking: Missing income
- E-commerce: Missing brand

Handling Imbalanced Dataset:

⇒ What problem does it solve?

- Misleadingly high accuracy
- Biased towards majority class
- Low Recall & F1-score

⇒ Why does it occur?

- Fraud is rare
- Rare events occur less frequently.

⇒ When to use imbalance handling?

- One class is smaller than another
- Minority class is important
- Accuracy is misleading
- For classification problems

⇒ When not to use it?

- Regression problems
- Business priorities
- Resampling before train-test split can cause data leakage.

⇒ Types of imbalance:

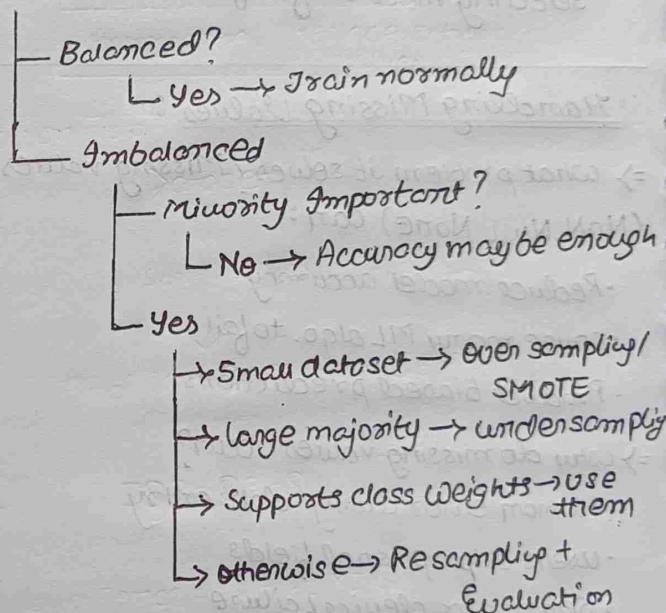
Type	Example	Best approach
Mild	600:400	Usually no special handling
Moderate	900:100	Class weights / Resampling
Severe	9900:100	SMOTE/Ensemble/ Cost-sensitive learning

⇒ Methods:

Method	Use when
Random under-sampling	Large majority class
Random over-sampling	Small minority class
Class weights	Algo. supports weights
Threshold tuning	Increase Recall
SMOTE	Create synthetic minority samples
Ensemble Methods	Severe imbalance

⇒ Decision flow:

Classⁿ Problem?



⇒ we can refer to following metrics as well:

- Precision
 - Recall
 - F1-score
 - ROC-AUC
 - PR-AUC
 - Confusion Matrix
- (Accuracy + these → Perfect Combination)

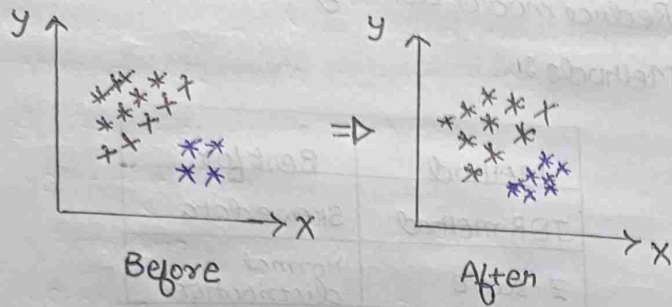
⇒ Real world Examples:

- Banking: Fraud detection → High Recall
- Healthcare: Cancer diagnosis → Avoid False negatives
- Manufacturing: Defect detection → Find rare defects

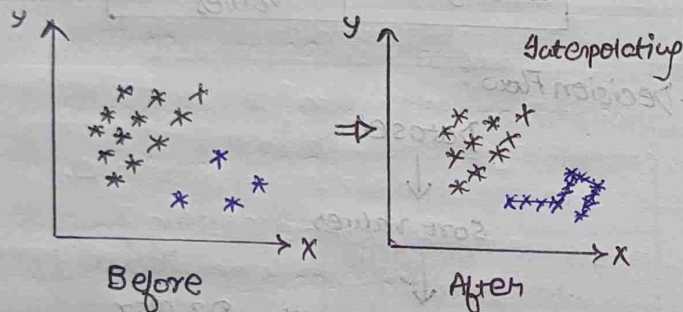
Handling Imbalanced Dataset

Using Smote :

⇒ What upsampling did wrong? Lack of Variance.



⇒ What SMOTE does? It creates new synthetic samples b/w existing minority-class points - it does not simply duplicate existing records.



⇒ What's SMOTE? Synthetic Minority Over-sampling Technique. It balances an imbalanced dataset by creating synthetic (new) minority class samples instead of duplicating existing ones.

⇒ What problem does it solve?

- Poor Recall
- Biased predictions
- Better minority class learning.

⇒ What was the issue with Random over Sampling?

- Overfitting (as no new info. is introduced)
- Increased false positive
- Outliers, errors or noise can be increased

⇒ When to use it?

- Classification problem
- Dataset is imbalanced
- want higher Recall & F1-score

⇒ When not to use it?

- Regression problem
- Before Train-Test split
- on Test/validation data

⇒ How SMOTE works?

- ① Pick a minority sample
- ② Find k nearest minority neighbors
- ③ Choose a neighbour
- ④ Generate a new point b/w them.

Formula:

$$\text{New sample} = x_i + \lambda(x_{\text{neighbour}} - x_i)$$

where,

x_i = original minority sample

$x_{\text{neighbour}}$ = one of its nearest minority neighbour

λ = a random no. b/w 0 and 1

e.g.,

$$x_i = (2, 2), x_{\text{neighbour}} = (4, 4), \lambda = 0.5$$

$$\therefore \text{New sample} = (2, 2) + 0.5((4, 4) - (2, 2)) \\ = (3, 3)$$

⇒ SMOTE relies entirely on K-NN internally to generate its synthetic data.

⇒ Types of SMOTE:

Type	Best Use
Regular SMOTE	General Purpose
Borderline SMOTE	class overlap
SVMSMOTE	Complex decision boundaries
KMeans-SMOTE	Complex minority distribution
SMOTENC	Numerical + Categorical data

⇒ Algorithm Compatibility:

- Logistic Regression
- SVM, KNN
- Decision tree
- XGBoost etc.

Handling outliers using Python :

⇒ What problem does 5 Number summary solves? It helps us:

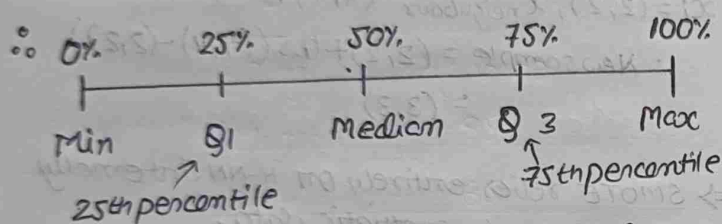
- understand the spread of data
- Detect outliers
- Build a box plot

⇒ When to use it?

- Before outlier detection
- During EDA
- Before feature engineering

⇒ The five numbers:

Statistic	Meaning
Minimum	Smallest value (25% of data lies below this value)
Q1 (25%)	First Quartile
Median (Q2)	Middle value (50%) (75% of data lies below this value)
Q3 (75%)	Third Quartile
Maximum	Largest value

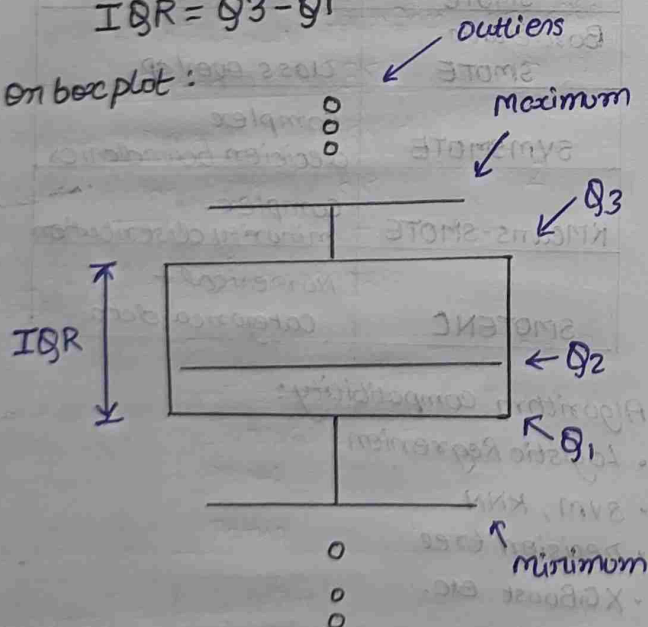


⇒ Interquartile Range (IQR): The IQR measures the spread of the middle 50% of the data.

Formula:

$$IQR = Q3 - Q1$$

⇒ On box plot:



⇒ Calculating Min and Max:

$$\text{Lower bound} = \text{Min} = Q1 - 1.5 \times IQR$$

$$\text{Upper bound} = \text{Max} = Q3 + 1.5 \times IQR$$

⇒ IQR is preferred over the total range because it is resistant to outliers.

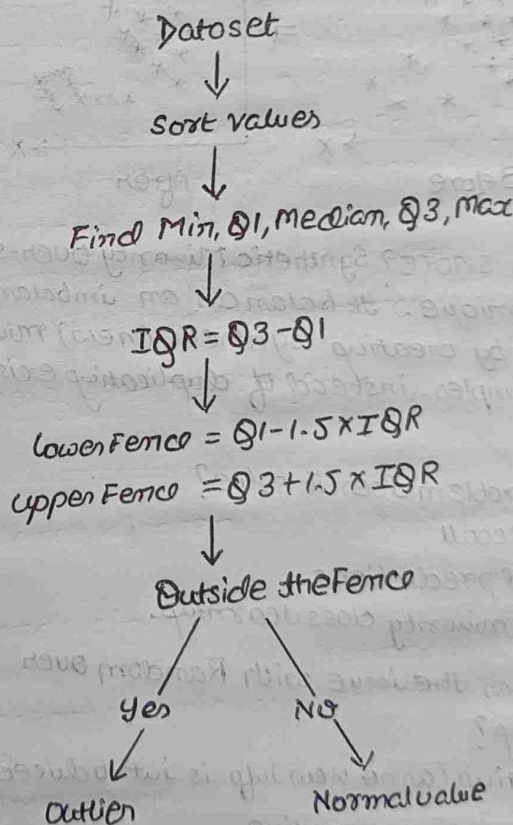
⇒ Problems caused by outliers:

- distort the mean
- Inc. variance
- Reduce model accuracy

⇒ Methods:

Method	Best for
IQR method	Skewed data
Z-score	Normal distribution
Box plot	Visualization
Percentile capping	Extreme values
Winsorization	Replace extreme values

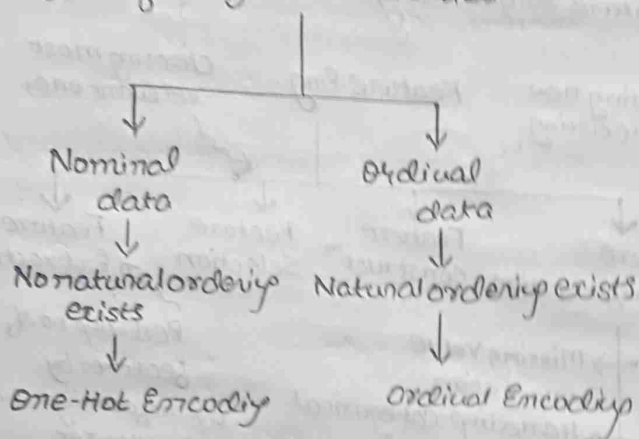
⇒ Decision Flow:



Data Encoding - Nominal or OHE:

⇒ What is data encoding? It's the process of converting categorical (text) data into numerical values so that ML algo. can understand and process it.

⇒ To perform mathematical operations we need Numerical data. Encoding converts that.



⇒ Types of data encoding:

Encoding	Best used for
Label Encoding	Target labels
Ordinal Encoding	ordered categories
One-Hot Encoding	Nominal categories
Binary Encoding	High cardinality features
Frequency Encoding	"
Count Encoding	category frequency
Target Encoding	High cardinality (carefully)
Hash Encoding	Very large datasets

⇒ Nominal Encoding: It assigns a unique integer to every category.

e.g.,

original:

Color
Blue
Green
Red

Encoded:

Color	Encoded
Blue	0
Green	1
Red	2

⇒ Mathematical Repⁿ of Nominal encoding:

if $C = \{\text{Blue, Green, Red}\}$

For mapping funⁿ,

$$f: C \rightarrow N$$

where, $N = \{0, 1, 2\}$

$$\therefore f(\text{Blue}) = 0$$

$$f(\text{Green}) = 1$$

$$f(\text{Red}) = 2$$

Problem: Model assumes
 $\text{Red} > \text{Green} > \text{Blue}$
which is not true.

⇒ One Hot Encoding: It converts every category into a separate binary colⁿ.
Each category receives:

1 → Present

0 → Absent

e.g.,

original:

color

Red

Blue

Green

Encoded:

	Red	Blue	Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1

⇒ Math Repⁿ of OHE:

Suppose C_j is the j th category.

Then,

$$\text{onehot}(x_i) = \begin{cases} 1, & x_i = C_j \\ 0, & \text{otherwise} \end{cases}$$

Meaning

• Position of the category → 1

• All other positions → 0

e.g.,

if Red = third category

then,

$$\text{onehot}(\text{Red}) = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

⇒ OHE is preferred for Nominal data bcz it doesn't create any fake relationship.

⇒ In OHE, categories are completely ind. it means one-hot vectors are orthogonal.

e.g.,

$$A = (1, 0, 0), B = (0, 1, 0)$$

$$\therefore A \cdot B = 0$$

Label and Ordinal Encoding:

⇒ Ordinal Encoding: used only when a natural order exists.

⇒ Label Encoding: used for no natural order.

Target Guided Ordinal Encoding:

⇒ categories are ordered according to how strongly they influence the target variable.

e.g.,

Target variable

Dataset City	Price
Mumbai	90
Mumbai	80
Delhi	70
Delhi	60
Patna	30
Patna	40

Step 1: Calc. mean target

$$\text{Mumbai} = \frac{90+80}{2} = 85$$

$$\text{Patna} = \frac{30+40}{2} = 35$$

$$\text{Delhi} = \frac{70+60}{2} = 65$$

City	Avg. Price
Mumbai	85
Delhi	65
Patna	35

Step 2: Sort Categories → High target mean gets the highest rank.

City	Mean Price	Encoding
Patna	35	0
Delhi	65	1
Mumbai	85	2

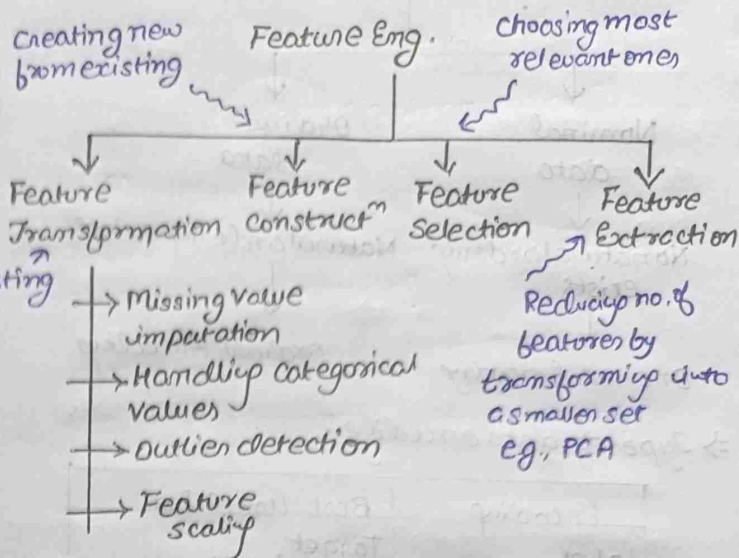
Step 3: Replace Categories.

City	Encoded
Mumbai	2
Delhi	1
Patna	0

⇒ when to use it?

- Categorical features has many unique categories.
- Category has a relationship with target.

Feature Engineering:



Feature Scaling - Standardization:

⇒ What is Feature scaling? It's a technique to standardize the independent features present in the data in a fixed range.

⇒ Why do we need Feature scaling?

- To prevent features with larger values from dominating.
- To improve distance based algo.
- To improve model performance

⇒ Types of feature scaling:

Standardization

Normalization

⇒ Standardization:

It is a feature scaling technique that transforms numerical features so that they have:

① Mean = 0

② Standard deviation = 1

⇒ Mathematical formula:

$$Z = \frac{X - \mu}{\sigma}$$

Standardized value

original feature value

mean of the feature

std. deviation of the feature

e.g.,

Age	Salary
20	30,000
30	50,000
40	70,000
50	90,000

$$\therefore H_{age} = 35$$

$$H_{salary} = 60000$$

Now,

$$\sigma_{age} = \sqrt{125} = 11.18$$

$$\sigma_{salary} = \sqrt{500000000} = 22360.68$$

Now,

For age = 20

$$z = \frac{20 - 35}{11.18} = -1.34$$

For salary = 30000

$$z = \frac{30000 - 60000}{22360.68} = -1.34$$

Final dataset:

Standardised age	Standardised salary
-1.34	-1.34
-0.45	-0.45
0.45	0.45
1.34	1.34

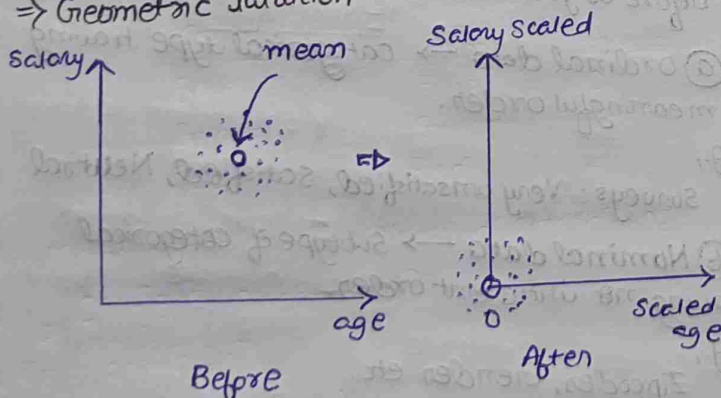
Now,

$$H_{std age} = 0 \quad \sigma = 1$$

$$H_{std salary} = 0 \quad \sigma = 1$$

$\Rightarrow z =$ No. of std. deviation away from the mean.

$\Rightarrow$ Geometric intuition:



$\Rightarrow$ Impact of outlier: Standardization does not remove outliers.

Feature Scaling - Normalization:

$\Rightarrow$ What is Normalization? The goal of normalization is to change the values of numeric columns in the dataset to use a common scale, without distorting differences in the ranges of values or losing information.

Most commonly used

$\Rightarrow$ Types of Normalization:

- Min-Max scaling
- Mean Normalization
- Max absolute
- Robust scaling

$\Rightarrow$ If standardization exists then why Normalization?

Standard. don't have any range or boundary.

$\Rightarrow$ Min-Max Scaling: It transforms data into a fixed range, usually $[0, 1]$.

Formula:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Output range: $[0, 1]$

e.g.,

Age	$X_{min} = 20, X_{max} = 50$
20	$X_{max} - X_{min} = 30$
30	
40	
50	

$\therefore$ for age = 20

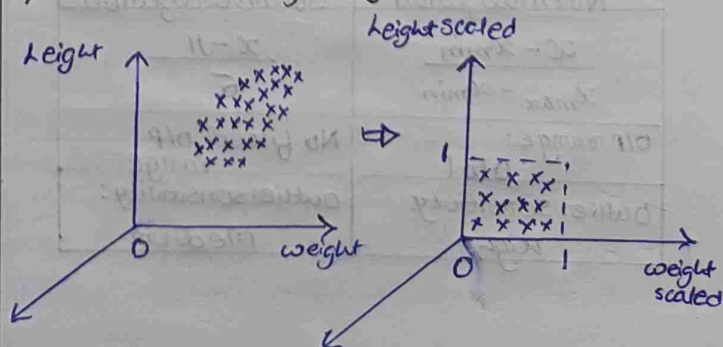
$$X_{scaled} = \frac{20 - 20}{30} = 0$$

and so on...

Final result:

Original	Normalized
20	0.00
30	0.33
40	0.67
50	1.00

$\Rightarrow$ Min-Max scaling on graph:



⇒ Advantages of Min-Max scaling:

- Neural Networks
- KNN
- Converts all features to a common scale.

⇒ Disadvantages of min-max:

- Highly sensitive to outliers. As the outlier will inc. the value of X_{max} leading to squeezed results.

⇒ Mean Normalisation: It centers the data around the mean and scale it by the feature's range. Formula used:

$$X_{scaled} = \frac{X - \mu}{X_{max} - X_{min}}$$

where, μ = mean of the feature

⇒ Max absolute (maxAbs Scaling): scales data by dividing every value by the max. absolute value. Formula used:

$$X_{scaled} = \frac{X}{|X_{max}|}$$

where,

$$|X_{max}| = \max(|X|)$$

$$O/P \text{ Range} \rightarrow [-1, 1]$$

⇒ Robust scaling: It's least affected by outliers. Formula:

$$X_{scaled} = \frac{X - \text{Median}}{IQR}$$

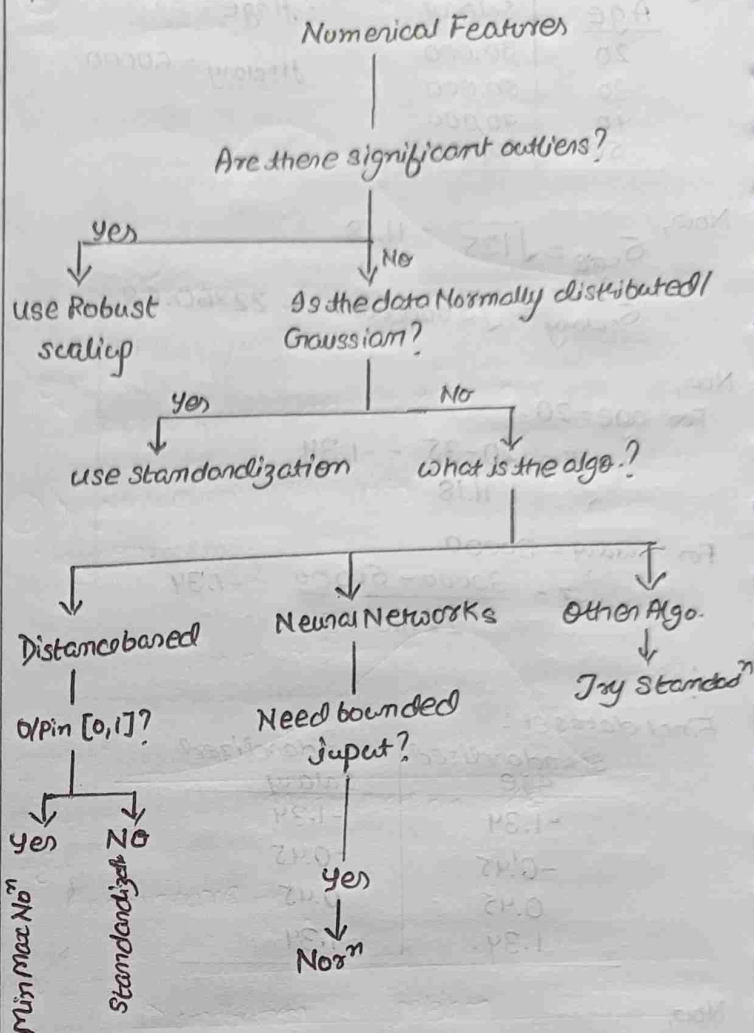
where, $IQR = Q3 - Q1$

Also, Robust scaling is a feature scaling technique, but not a normalization technique.

⇒

Normalization	Standardization
$\frac{X - X_{min}}{X_{max} - X_{min}}$	$\frac{X - \mu}{\sigma}$
O/P range: 0 to 1	No fixed O/P range
Outlier sensitivity: High	Outlier sensitivity: Medium

⇒ Flow chart:



⇒ Tip: Most of the time standardization will be used.

Column Transformer:

⇒ Before it just know:

① Ordinal data → categorical type having a meaningful order.

e.g.,

Surveys: Very unsatisfied, satisfied, Neutral

② Nominal data → subtype of categorical having no inherent order.

e.g.,

Zipcodes, Gender etc.

⇒ What is Column Transformer? It's a scikit-learn object that applies different preprocessing pipelines to different subsets of columns their outputs into one transformed dataset.

⇒ Why do we need it? Real world datasets have different types of features. And thus, one kind of encoding can't be applied to each column. Instead, it allows to scale the numeric colⁿ, encode the categories etc. simultaneously.

e.g.,

Age	Salary	Gender	City
25	50000	Male	Delhi

↑ Numeric ↑ categorical ↑ categorical

⇒ What problem does it solve?

- Repeated code
 - Hard to maintain
- } colⁿ transformers automates everything.

⇒ Intuition: different columns go to different transformers. Every feature gets its needed feature treatment.

⇒ How does it work?

Step 1: Select numeric colⁿ → apply scaling

Step 2: " categorical colⁿ → apply OHE

Step 3: concatenate all the transformed.

⇒ Alternatives:

- Pipeline
- FeatureUnion (older approach)
- Manual preprocessing

⇒ We should avoid `fit()` on test data as it leads to data leakage.

⇒ Formula:

For feature matrix X :

$$X' = [T_1(X_{num}), T_2(X_{cat}), \dots, T_K(X_{subset})]$$

Where,

T_i is the transformer applied to a subset of columns, and the outputs are concatenated horizontally.

Machine Learning Pipeline

⇒ What is it? It's a chain of transformers followed by a final estimator, where each step receives the transformed output from the previous step.

⇒ Suppose X is the original dataset. A pipeline consists of $f_1, f_2, \dots, f_n$ where each f_i transforms the data. The final model g uses the transformed data. **Mathematical Explanation**

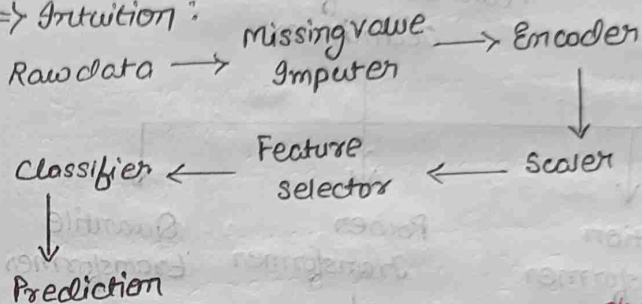
Hence,

$$\hat{y} = g(f_n(f_{n-1}(\dots f_2(f_1(X)))))$$

⇒ without pipeline:

- Repeated code
- Wrong order
- Difficult deployment

⇒ Intuition:



[Every transformer receives the transformed dataset from the previous transformer]

e.g.,

Age	Salary	Gender
25	50000	Male
NaN	80000	Female
30	4000	Male

Step 1: Missing value imputer

i/p: 25	O/P: 25
NaN	27.5
30	30

Step 2: Encoder

i/p: Male	O/P: Male	Female
Female	1	0
Male	0	1

Step 3: scaler

i/p: 25	O/P: -1.2
27.5	0.0
30	1.2

Step 4: Feature selection

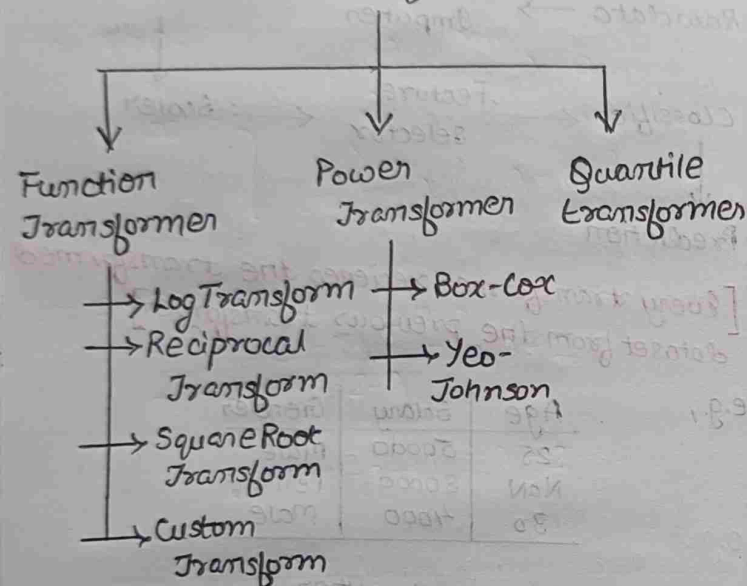
Step 5: Classifier Receives the processed data & produces survived / Not survived.

⇒ Core Components :

- ① Transformer → Modifies data
- ② Estimator → Learns & predict
- ③ Pipeline → chain steps sequentially
- ④ Column Transformer → applies different preprocessing to diff. columns.
- ⑤ Feature Union → Runs multiple transformers in parallel and combines their outputs.

Data Transformation Techniques :

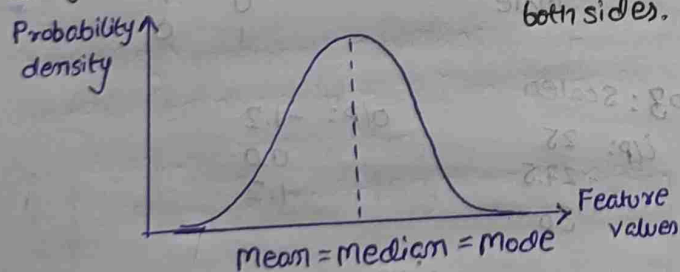
⇒ Feature Transformation Techniques



⇒ Let's first talk about 'skewness' so, what is skewness? It's a key statistical measure that shows how data is spread out in a dataset.

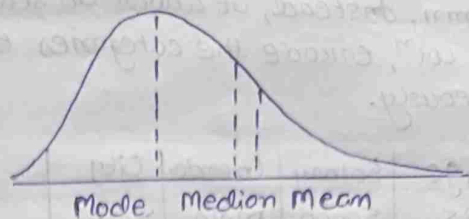
⇒ Types of skewness :

- ① Symmetric (No skewness) → data is evenly distributed on both sides.



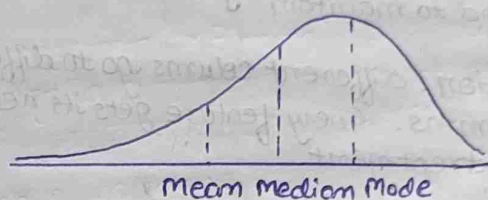
e.g., Adult heights, IQ scores (approx)

- ② Positive skewness (Right skewed) → Right tail is longer.



e.g., salaries, stock market return

- ③ Negative skewness (Left skewed) → left tail is longer.



e.g., Exam score

⇒

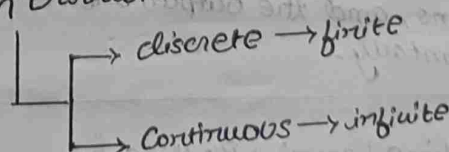
Distribution	mean	Skewness
Symmetric	Mean = Median = Mode = 0	
+ve skew	Mean > Median > Mode	> 0
-ve skew	Mean < Median < Mode	< 0

⇒ skewness formula :

$$\text{Skewness} = \frac{E\{(X - \mu)^3\}}{\sigma^3}$$

where; X = random variable
 μ = mean
 σ = std. deviation
 E = average

⇒ Random Variable

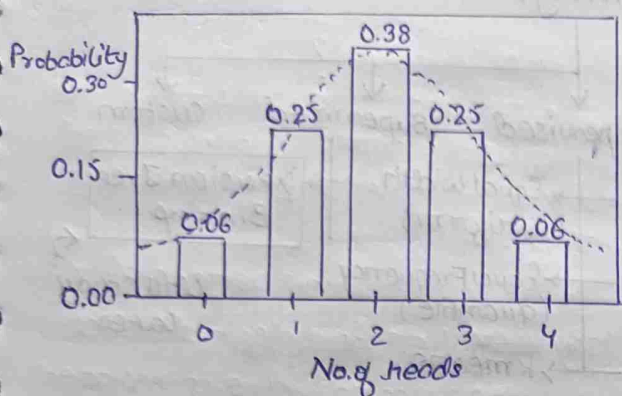


⇒ Prob. of discrete random variable is Probability Mass Function.

e.g., Tossing 4 coins

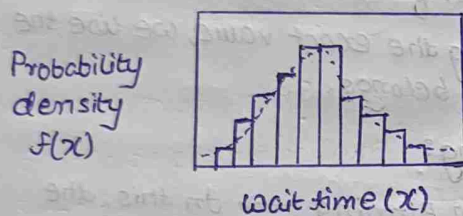
$$\therefore P(0H) = \frac{1}{16}, P(1H) = \frac{4}{16}, P(2H) = \frac{6}{16}$$

$$P(3H) = \frac{4}{16}, P(4H) = \frac{1}{16}$$



⇒ Probability density Function is calculated for Continuous random variable.

- Prob. density is never negative
- entire area under graph is 1



⇒ Log transform: each value of a feature is replaced by its logarithm.

Instead of using the original value x , we use:

$$\log(x) \text{ or } \log(1+x)$$

e.g.,

Salary	⇒	Salary transformed
10,000		9.21
20,000		9.90
30,000		10.82
5,00,000		15.42

⇒ Common objectives of function transformations:

- Reduce skewness
- Make distⁿ closer to normal
- Reduce effects of outliers
- Stabilize variance
- Make algo. easier to use on normally distributed features

⇒ Positive (Right) skewed → Log, sq. root, Reciprocal, Box-Cox, Yeo-Johnson

Negatively (left) skewed → Square, Cube, Yeo-Johnson

⇒ Tree based models are generally not much affected by skewed features.

⇒ Reciprocal transform: to be avoided if feature has 0 as its value.

$$x' = \frac{1}{x}$$

⇒ Square Root Transform:

$$x' = \sqrt{x}$$

⇒ Strength:

$$\text{SquareRoot} < \text{Log} < \text{Reciprocal}$$

Power Transformer:

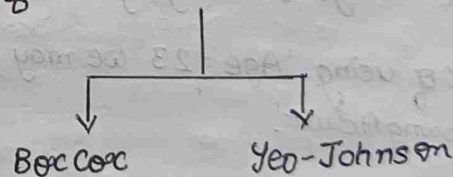
⇒ What is power transformation? It's a family of mathematical transformations that apply an exponent (power) to the data in order to make it more suitable for ML algorithms.

⇒ Unlike simple transformations (log, square root, reciprocal), a power transformation automatically finds the most suitable transformation for our data.

⇒ Goal of Power transformation:

- Reduce skewness
- Make the distribution closer to a normal distribution.
- Reduce effects of outliers
- Stabilize variance

⇒ Types of Power Transfⁿ:



⇒ Box-Cox Transformation:

$$x' = \begin{cases} \frac{x^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(x), & \lambda = 0 \end{cases}$$

where, x = original value

x' = transformed value

λ = transformation parameter chosen automatically

⇒ How does box-cox works? For any skewed feature, the algo tries many λ values and then it measures which transformed data is closest to a normal distribution. That λ is selected automatically.

⇒ Disadvantages of box-cox:

- can't handle zero
- can't handle negative values

⇒ What can handle these disadvantages?
Yeo-Johnson Transformation.

⇒ Yeo-Johnson Transformation:

For $x \geq 0$:

$$x' = \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(x+1), & \lambda = 0 \end{cases}$$

For $x < 0$:

$$x' = \begin{cases} -\frac{(-x+1)^{2-\lambda} - 1}{2-\lambda}, & \lambda \neq 2 \\ -\ln(-x+1), & \lambda = 2 \end{cases}$$

Encoding Numerical values:

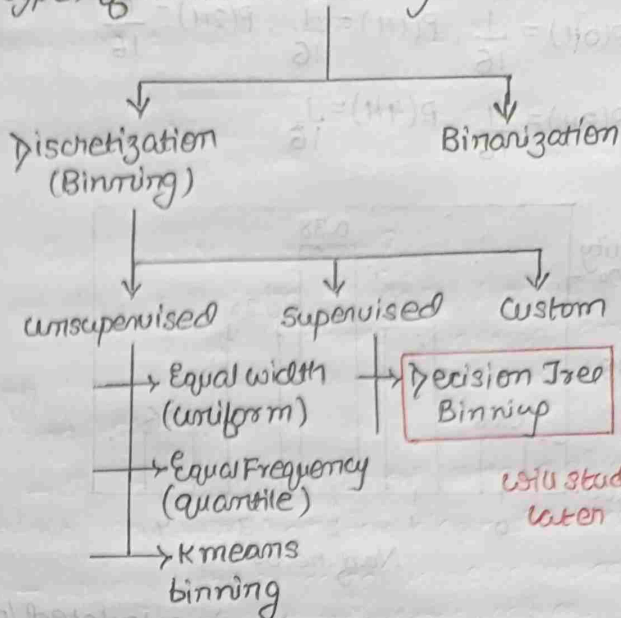
⇒ Why to encode numerical values? Sometimes the exact numerical value is less important than the range or category it belongs to.

e.g.,
Instead of using 'Age = 23' we may use 'Age = Young Adult'.

⇒ Goal of Numerical encoding:

- Reduce noise in numerical features
- Simplify complex data.
- Improve performance of some ML algo..

⇒ Types of Numerical encoding:



⇒ Discretization (or Binning): It's the process of converting a continuous numerical feature into a finite no. of intervals (bins).

Instead of using the exact value, we use the bin to which it belongs.

⇒ Types of Binning:

(A) unsupervised Binning → In this, the target variable is not used while creating bins, only the feature value determines the interval. Some common methods are:

(i) Equal width (uniform) binning: The feature's range is divided into intervals of equal width.

Where,

$$\text{Bin width} = \frac{\text{Maximum} - \text{Minimum}}{\text{No. of bins}}$$

e.g.,

Age	
10	
11	
12	
⋮	
90	

∴ Range = Max - Min
= 90 - 10
= 80

and if no. of bins = 4

$$\therefore \text{Bin width} = \frac{80}{4} = 20$$

∴ Bins become: 10-30

30-50

50-70

70-90

Each interval has the same width

∴ disadvantages:

- sensitive to outliers
- some bins may contain many observations, while other contain very few.

(i) Equal Frequency (Quantile) Binning:

In this the data is divided into bins such that each bin contains approx the same no. of observations.

The boundaries of the bins are determined by quantiles (percentiles).

e.g., age distribution of 1000 people with 10 bins,

$\therefore 1000 \div 10 = 100$ ($\therefore$ each bin shall contain 100 people or 10% of the data)

Step 1: Sort the data

0, 1, 2, 3, 4, ..., 74

Step 2: Find the deciles, assume the percentiles are:

10th percentile = 16

20th " = 20

200th " = 74

These percentile values become the bin boundaries.

Step 3: Create the bins

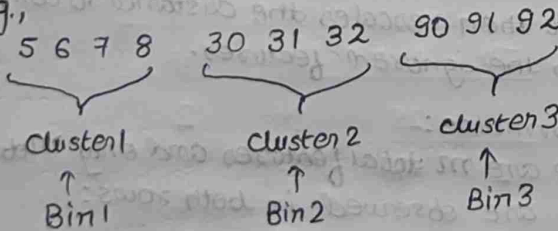
Bin	Age Interval	Percentage of data
Bin 1	0-16	10%
Bin 2	16-20	10%
...	...	...
Bin 10	50-74	10%

$\therefore$ disadvantages:

- Bin width becomes irregular

(ii) K-means Binning: It groups similar numerical values into clusters, and each cluster becomes a bin. It uses K-means algo., which groups nearby values based on distance.

e.g.,



advantages:

- better than fixed width bins

- adapts to the natural structure of the data.

disadvantages:

- More computationally expensive

- Result depends on the chosen no. of clusters.

Method	Interval width	No. of Points Per Bin	Best for
(i)	same	Different	uniformly distributed data
(ii)	Different	Same	skewed data
(iii)	variable	variable	Naturally clustered data

(B) Supervised Binning $\rightarrow$ later on!!

(C) Custom Binning: we manually define the bin boundaries using domain knowledge.

$\Rightarrow$ Binarization: It converts a numerical feature into only two possible values, 0 and 1, based on a threshold.

For a threshold T ,

$$x' = \begin{cases} 1, & x > T \\ 0, & x \leq T \end{cases}$$

e.g., Original

20

45

50

60

85

if threshold is 50

After Binarization

0

0

0

1

1

disadvantages:

- loses a lot of information

- small changes around the threshold can flip the encoded value.

Handling Mixed Variables:

$\Rightarrow$ what are mixed variables? It's a feature that contains both numerical and categorical info. within the same feature.

e.g., Cabin No.

C22

E85

B30

A20

Here, C, E, B, A $\rightarrow$ Cabin deck (Categorical)

85, 22, 30, 20 $\rightarrow$ Cabin No. (Numerical)

$\Rightarrow$ Issue with mixed variables:

• Hide useful patterns

• Can't be directly encoded or scaled.

• Make preprocessing difficult

⇒ How do we handle Mixed variables?
Split that column into multiple columns,
one for categorical part and other for
numerical part.

e.g.,

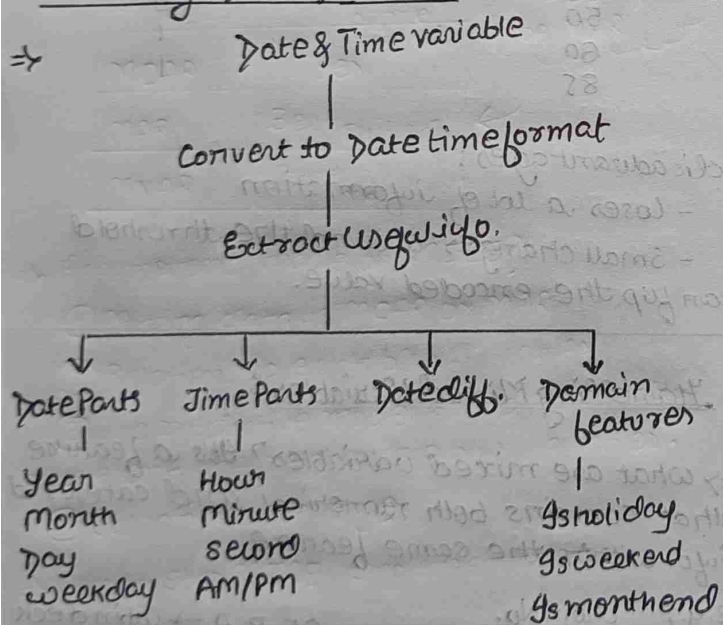
①

Cabin	Deck	Cabin No.
C80	C	80
D30	D	30
E44	E	44
C75	C	75

② Column with mixed data types.

col-name	No.	Text
7	7	NA
3	3	NA
1	1	NA
A	NA	A
C	NA	C
8	8	NA

Handling Date & Time :



Multivariate Imputation :

⇒ What is Multivariate imputation? In
this the missing values of one feature
are estimated using the information
from multiple other features.

⇒ Types of multivariate imputation:

KNN Imputer Iterative Imputer

⇒ What is KNN imputer?

— It fills missing values by finding the
K most similar observations (neighbors) and
using their known values to estimate the
missing values.

— Similarity is measured using a distance
metric (say ~~mean~~ Man Euclidean distance)

— usually, the missing value is replaced
by the mean of the neighbor's values.

⇒ workflow:

dataset with missing values

↓
Find K-nearest Rows

↓
Take their known values

↓
Average them

↓
Replace missing value.

⇒ How the distance is calculated?

— Man Euclidean distance is used

— " " " calculates the
distance using only the feature that are
available in both rows while ignoring missing
values.

— It then rescales the distance to compen-
-sate for the ignored features.

— Formulae:

If there are m total features and only m_obs
features are observed in both rows:

$$d(x, y) = \sqrt{\frac{m}{m_{obs}} \sum_{i \in observed} (x_i - y_i)^2}$$

where, $\frac{m}{m_{obs}}$ is the scaling factor

for the missing dimensions.

e.g.,

	Height	Weight
A	160	55
B	165	60
C	170	? ← missing
D	175	70
E	180	75

For $K=2$

Step 1: Compute Norm Euclidean distance

$\text{dist}(C, A)$

$m=2, m_{\text{obs}}=1$

$$\therefore d = \sqrt{\frac{2}{1}(170-160)^2} = 14.14$$

$\text{dist}(C, B)$

$m=2, m_{\text{obs}}=1$

$$\therefore d = \sqrt{\frac{2}{1}(170-165)^2} = 7.07$$

$\text{dist}(C, D)$

$$\therefore d = 7.07$$

$\text{dist}(C, E)$

$$\therefore d = 14.14$$

Step 2: $\because K=2 \rightarrow$ select two nearest neighbour

Student Distance

B 7.07

D 7.07

A 14.14

E 14.14

$\therefore B$ and D be two nearest neighbour

Step 3: average their values

Weight

60

$$\therefore \text{mean} = \frac{60+70}{2} = 65$$

70

$\therefore$ missing weight is imputed as 65 kg.

(ii)

S.No	Feature 1	Feature 2	Feature 3	Feature 4
1	33	---	67	21
2	---	45	68	12
3	23	51	71	18
4	40	---	81	---
5	35	60	79	---

For $K=2$,

From (1),

$$d = \sqrt{\frac{3}{2}((68-67)^2 + (12-21)^2)} = 11.29$$

From (2),

$$d = \sqrt{\frac{3}{3}((51-45)^2 + (71-68)^2 + (18-12)^2)} = 9$$

From (4),

$$d = \sqrt{\frac{3}{1}(31-68)^2} = 5.19$$

From (5),

$$d = \sqrt{\frac{3}{2}((60-45)^2 + (79-68)^2)} = 14.79$$

clearly, (3) & (4) are nearest

$$\therefore \text{mean} = \frac{40+23}{2} = 31.5 \text{ (ans)}$$

Feature 1 of (2)

$\Rightarrow$ Advantages of KNN imputer:

• uses Relationship b/w features

• More accurate

• No assumption about data distribution

$\Rightarrow$ Disadvantages:

• Computationally expensive

• Sensitive to feature scaling

• Choosing K is important

$\Rightarrow$ why can't we use normal euclidean distance?

Age Salary

25

?

30

40

35

50

$\therefore$ euclidean distance is

$$\sqrt{(25-30)^2 + (7-40)^2}$$

But,

?-40 can't be computed bcz value is missing.

$\therefore$ Normal Euclidean distance can't be used.

Multivariate Imputation By Chained

Equations (MICE):

⇒ What is MICE?

- It is an iterative, model based imputation technique that predicts missing values using other variables in the dataset.

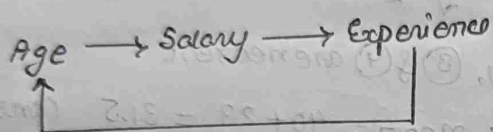
- The process is repeated multiple times until the imputed values stabilize.

⇒ Why is it called "Chained Equations"?

For a dataset having three features with missing values:

- Age
- Salary
- Experience

MICE predicts them in a chain:



After predicting experience, it returns to age again. The cycle continues until the values converge, hence named so.

⇒ Keep in mind:

Missing data type	Can MICE handle?
MCAR	Yes
MAR	Best suited
MNAR	Generally No

Best choice is MICE if data be MAR

⇒ Assumptions made by MICE?

- ① Missing data is MCAR or MAR
- ② Relationship exists b/w variables
- ③ The predictive model should reasonably represent the relationship b/w variables.
- ④ Iterations converge

⇒ Step by step MICE Implementation:

ID	Age	Salary	Experience
1	25	30	2
2	30	45	5
3	35	NaN	8
4	40	70	NaN
5	NaN	90	15

Step1: Initial imputation

$$\text{Mean age} = \frac{25+30+35+40}{4} = 32.5$$

$$\text{Mean Salary} = \frac{30+45+70+90}{4} = 58.75$$

$$\text{Mean Experience} = \frac{2+5+8+15}{4} = 7.5$$

∴ temporary dataset

ID	Age	Salary	Experience
1	25	30	2
2	30	45	5
3	35	58.75	8
4	40	70	7.5
5	32.5	90	15

Step2: Predict age...

Now, treat Age as the target variable.

Model:

$$\text{Age} = f(\text{Salary}, \text{Experience})$$

Predict only Row 5. Suppose model predicts 34. So, replace 32.5 → 34

Step3: Predict salary

Now, salary becomes the target variable.

Model:

$$\text{Salary} = f(\text{Age}, \text{Experience})$$

Predict Row 3. Suppose 58.75 → 62

Step4: Predict Experience

$$7.5 \rightarrow 13$$

∴ dataset becomes:

ID	Age	Salary	Exp.
1	25	30	2
2	30	45	5
3	35	62	8
4	40	70	11
5	34	90	15

Step5: Repeat again... The process repeats until the values no longer change significantly.

⇒ Advantages:

- Uses Relationship b/w features
- Flexible
- Iteratives improves Estimates

⇒ Disadvantages:

- Computationally expensive
- Sensitive to model choice
- May not converge quickly

Feature Construction & Feature Splitting :

⇒ Feature Construction → Combine Existing features

↓
create new info.

↓
New features

⇒ Feature Splitting → Split one feature

↓
Extract hidden information

↓
Multiple features

Curse of dimensionality :

⇒ What it is? It refers to the problem that arise when the no. of features (dimensions) increases, causing the data to become sparse and making it difficult for ML algo. to learn meaningful patterns.

⇒ Suppose, each feature has only 10 possible values. Then,

Features	Possible Comb ⁿ
1	10
2	100
3	1000
4	10000
10	10,000,000,000

∴ Formula :

Possible combination
= K^d

where,

K = Possible values
per feature

d = no. of dimension
(features)

It's exponential growth causes sparsity.

⇒ Problems caused by the curse of dim :

- ① Data sparsity
- ② Distance becomes less meaningful
- ③ Overfitting
- ⑤ Higher Computational cost

⇒ How to overcome the curse of dim.?

- ① Feature Selection
- ② Dimensionality Reduction
- ③ Feature Engineering